

PHASE 6

FitBuddy- AI Fitness Plan Generator using Gemini Models

Project Testing

1. Introduction

Testing is an important phase in the development of the **FitBuddy – AI Fitness Plan Generator**. After completing the design, planning, and development activities, the application needs to be checked to ensure that its major components work together correctly.

FitBuddy combines several technologies and components, including **FastAPI, Google Gemini AI, SQLite, SQLAlchemy, Jinja2 templates, HTML, CSS, and Uvicorn**. Therefore, testing is required not only for individual features but also for the interaction between the frontend, backend, AI services, and database.

The FitBuddy source project describes local deployment and verification using a FastAPI development server. The application can be accessed through `http://127.0.0.1:8000`, while FastAPI's interactive documentation is available through `/docs` for endpoint testing.

The main purpose of this phase is to verify the complete user journey:

User Input → FastAPI → AI Processing → Database → Generated Plan → Feedback → Updated Plan

2. Purpose of Project Testing

The main purpose of testing FitBuddy is to determine whether the implemented system performs the functions identified during the requirement analysis and development phases.

Testing focuses on:

- Checking whether users can enter their required information.
- Checking whether FastAPI receives and processes user input.
- Checking whether the AI service generates a personalized workout plan.
- Checking whether the generated plan contains the expected workout structure.
- Checking whether nutrition or recovery advice is generated.
- Checking whether feedback can be submitted.
- Checking whether an existing workout plan can be updated.
- Checking whether user and workout information is stored.
- Checking whether original and updated plans can be retrieved.
- Checking whether the frontend displays generated information correctly.
- Checking whether the main application routes work.
- Checking whether the local application can be started successfully.
- Checking whether FastAPI interactive documentation is accessible.

The source project specifically describes Pydantic structures such as `UserInput` and `FeedbackRequest` for validating the structure of submitted information.

3. Testing Objectives

The following objectives are defined for FitBuddy testing.

Objective 1 – Functional Verification

Verify that each major feature performs the function for which it was developed.

Objective 2 – Input Validation

Verify that user information such as name, user ID, age, weight, fitness goal, and workout intensity can be processed correctly.

Objective 3 – AI Verification

Verify that the AI layer receives the appropriate user information and produces the expected type of response.

Objective 4 – Database Verification

Verify that user information and generated plans are stored and retrieved correctly.

Objective 5 – Feedback Verification

Verify that feedback can be used to generate an updated workout plan while preserving the original plan.

Objective 6 – Frontend Verification

Verify that the Jinja2 templates display user information, workout plans, nutrition tips, and feedback functionality correctly.

Objective 7 – Integration Verification

Verify that the frontend, backend, AI layer, and database operate together as one application.

Objective 8 – Deployment Verification

Verify that FitBuddy starts correctly on the local development server and can be accessed through a web browser.

4. Testing Strategy

FitBuddy testing can be organized into several levels.

Testing Levels

1. **Unit-level testing**
2. **Functional testing**
3. **Integration testing**

4. **Frontend/UI testing**
5. **AI response testing**
6. **Database testing**
7. **Route/API testing**
8. **Local deployment testing**
9. **End-to-end testing**

Figure 1: FitBuddy Testing Workflow

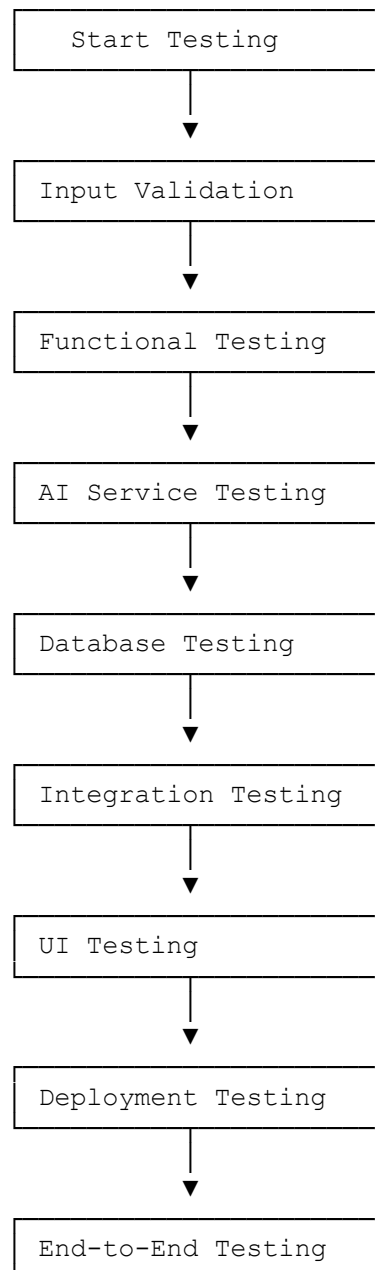


Figure 1: FitBuddy Testing Workflow

5. Test Environment

The testing environment follows the technologies specified in the FitBuddy source project.

Software Environment

Component	Technology
Programming Language	Python
Backend Framework	FastAPI
Server	Uvicorn
AI Service	Google Gemini
Database	SQLite
ORM	SQLAlchemy
Frontend	HTML/CSS
Template Engine	Jinja2
API Testing	FastAPI /docs
Development Environment	VS Code / Python environment

The source document specifies creation of a virtual environment and installation of dependencies including FastAPI, Uvicorn, Jinja2, SQLAlchemy, python-multipart, and the Google Generative AI SDK.

6. Functional Testing

Functional testing verifies whether FitBuddy performs the functions defined for the application.

The major functions are:

- User input collection
- Workout plan generation
- Nutrition tip generation
- Feedback submission
- Workout plan updating
- User storage
- Plan storage
- Plan retrieval
- Administrative user viewing

7. User Input Testing

The FitBuddy homepage collects information required for generating a personalized plan.

The documented input fields include:

- Name
- User ID
- Age

- Weight
- Fitness Goal
- Workout Intensity

The source project describes fitness goals such as weight loss and muscle gain and intensity levels such as low, medium, and high.

Test Conditions

The following conditions should be checked:

Test Case UI-01 – Valid User Information

Input: Valid name, user ID, age, weight, goal, and intensity.

Expected Result:

The application accepts the information and sends it to the workout-generation process.

Test Case UI-02 – Fitness Goal Selection

Input: Weight loss, muscle gain, or another supported goal.

Expected Result:

The selected goal is passed to the backend and used in AI generation.

Test Case UI-03 – Workout Intensity

Input: Low, medium, or high intensity.

Expected Result:

The selected intensity is passed to the backend.

Test Case UI-04 – Missing Information

Input: One or more required fields are left empty.

Expected Result:

The application should prevent or appropriately handle incomplete input rather than generating an invalid plan.

8. Workout Plan Generation Testing

Workout generation is one of the main functions of FitBuddy.

The documented function `generate_workout_gemini()` uses user information such as fitness goal and intensity to generate a personalized 7-day workout plan. The documented output includes warm-up, main workout details, and cooldown or recovery guidance.

Test Case WG-01 – Workout Generation

Input: Valid user information.

Expected Result:

A personalized workout plan is generated and displayed on the result page.

Test Case WG-02 – Seven-Day Structure

Input: Valid fitness profile.

Expected Result:

The generated response should provide a day-wise workout plan.

Test Case WG-03 – Goal-Based Generation

Input: Weight-loss goal.

Expected Result:

The generated plan should be related to the selected goal.

Test Case WG-04 – Intensity-Based Generation

Input: Low, medium, or high intensity.

Expected Result:

The selected intensity should be considered during workout generation.

Test Case WG-05 – Workout Details

Expected Result:

The workout response should contain exercise-related details such as exercises, sets, repetitions or duration, and rest information where provided by the generated response.

9. Nutrition and Recovery Tip Testing

FitBuddy also provides a nutrition or recovery tip alongside the workout plan.

The source describes `generate_nutrition_tip_with_flash()` as the function responsible for generating a concise nutrition or recovery recommendation related to the user's selected fitness goal.

Test Case NT-01 – Nutrition Tip Generation

Input: Valid fitness goal.

Expected Result:

A concise nutrition or recovery tip is generated.

Test Case NT-02 – Goal-Based Tip

Input: A particular fitness goal.

Expected Result:

The nutrition recommendation should relate to the selected goal.

Test Case NT-03 – Result Page Display

Expected Result:

The nutrition tip should be displayed together with the generated workout plan.

10. Feedback Testing

Feedback is an important feature because FitBuddy is designed to allow users to refine an existing workout plan.

The source describes examples such as asking for more cardio or adding yoga. The original plan and feedback are passed to the workout-update function, and the resulting updated plan is stored separately.

Test Case FB-01 – Feedback Submission

Input: Valid user ID and feedback.

Expected Result:

The feedback is accepted and processed.

Test Case FB-02 – Retrieve Original Plan

Input: Existing user ID.

Expected Result:

The application retrieves the corresponding original workout plan.

Test Case FB-03 – AI Plan Update

Input: Original plan + feedback.

Expected Result:

The AI service generates a revised workout plan based on the feedback.

Test Case FB-04 – Updated Plan Storage

Expected Result:

The updated plan is stored separately from the original plan.

Test Case FB-05 – Updated Result Display

Expected Result:

The revised workout plan is displayed to the user.

11. Database Testing

FitBuddy uses **SQLite with SQLAlchemy** for persistence. User information and generated plans are stored in the database, while feedback-based updates are maintained separately so that the original and updated versions can be reviewed.

Database Testing Areas

The following operations should be checked:

- Save user
- Save workout plan
- Retrieve user
- Retrieve original plan
- Update plan
- Retrieve all users
- Retrieve all plans

Test Case DB-01 – Save User

Expected Result:

User information is stored in SQLite.

Test Case DB-02 – Save Plan

Expected Result:

The generated workout plan is associated with the user.

Test Case DB-03 – Retrieve Plan

Expected Result:

The stored plan can be retrieved using the appropriate user information.

Test Case DB-04 – Update Plan

Expected Result:

The revised plan is stored without losing the original plan.

Test Case DB-05 – Multiple Users

Expected Result:

Information belonging to different users remains separately identifiable.

12. FastAPI Route Testing

The FitBuddy application uses FastAPI routes to connect the frontend, backend logic, AI functions, and database.

The documented routes include:

```
/
/generate-workout
/submit-feedback
/view-all-users
```

The source explains that / displays the homepage, /generate-workout processes user information and generates a plan, /submit-feedback updates an existing plan, and /view-all-users provides the administrative view.

Figure 2: Route Testing Structure

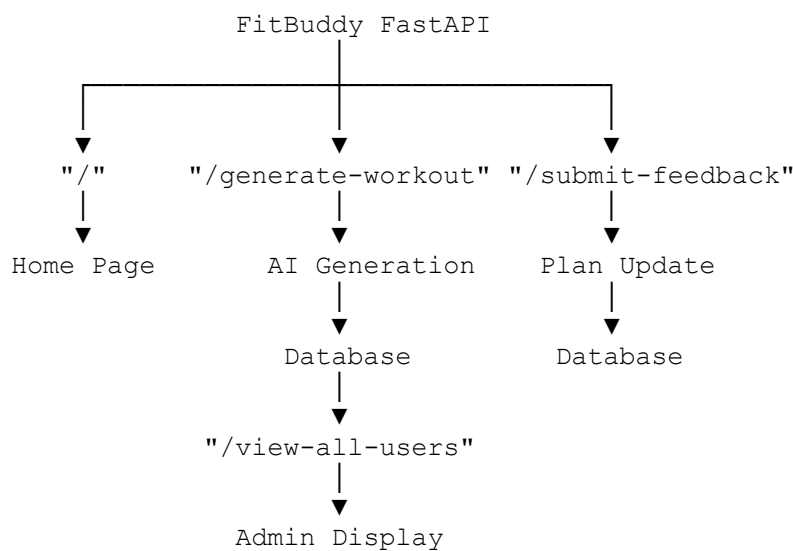


Figure 2: FastAPI Route Testing Structure

13. Home Route Testing

Route

```
/
```

Purpose

The home route displays the main FitBuddy input interface.

Expected Behavior

The homepage should load the input form and allow the user to enter the information required for generating a fitness plan.

14. Workout Generation Route Testing

Route

/generate-workout

Testing Flow

```
User Form
↓
FastAPI Form Data
↓
Pydantic Validation
↓
Workout AI Function
↓
Nutrition AI Function
↓
Database Storage
↓
result.html
```

Figure 3: Workout Generation Testing Flow

The source documents that the route receives form information, constructs the user input structure, calls the workout and nutrition functions, stores user and plan information, and returns the result template.

15. Feedback Route Testing

Route

/submit-feedback

Testing Flow

```
Existing User
↓
User ID
↓
Feedback
↓
Retrieve Original Plan
↓
AI Plan Update
↓
Save Updated Plan
↓
Display Updated Result
```

Figure 4: Feedback Testing Flow

This testing verifies the central adaptive feature of FitBuddy.

16. Admin View Testing

Route

`/view-all-users`

The source describes this page as displaying registered users and their plans. It allows the administrator or developer to review information including username, age, weight, goal, intensity, and original or updated workout plans.

Test Case ADM-01 – User List

Expected Result:

Available user records are displayed.

Test Case ADM-02 – User Information

Expected Result:

Relevant stored user information is displayed.

Test Case ADM-03 – Original Plan

Expected Result:

The original workout plan can be reviewed.

Test Case ADM-04 – Updated Plan

Expected Result:

An updated plan is displayed when the user has submitted feedback.

17. Frontend and UI Testing

Frontend testing checks whether the user can interact with the system correctly.

The documented templates are:

```
index.html
result.html
all_users.html
```

The source explains that `index.html` collects user information, `result.html` displays the workout plan and nutrition tip and accepts feedback, and `all_users.html` provides the administrative view.

UI Testing Areas

- Page loading
- Form visibility
- Input fields
- Buttons
- Result display
- Workout formatting
- Nutrition tip display
- Feedback form
- User information display
- Admin table
- Responsive layout

Figure 5: Frontend Testing Flow

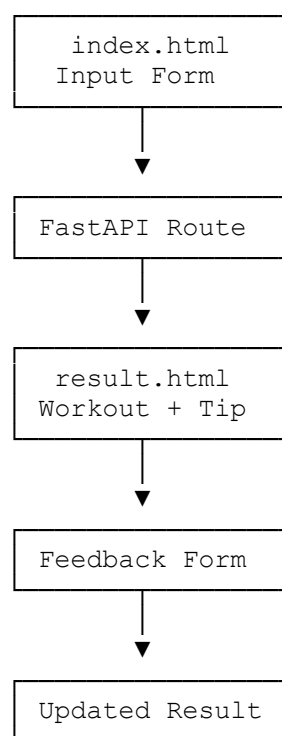


Figure 5: Frontend Testing Flow

18. Jinja2 Template Testing

FitBuddy uses Jinja2 to dynamically display backend-generated information.

The source explains that FastAPI's `Jinja2Templates` is used to render the templates and pass dynamic context to HTML pages. The result page uses values such as username, goal, intensity, workout plan, and nutrition tip.

Test Conditions

The following should be verified:

- Correct user information appears on the result page.
- Workout plan is inserted into the appropriate result section.
- Nutrition tip is displayed.
- Feedback form is available.
- User records are correctly looped through on the admin page.
- Missing values do not unexpectedly break the page

19. AI Response Testing

AI-generated output requires special attention because the response is generated dynamically.

The documented FitBuddy AI functions are:

```
generate_workout_gemini()  
generate_nutrition_tip_with_flash()  
update_workout_plan()
```

These functions are responsible for workout generation, nutrition/recovery guidance, and feedback-based workout updates.

AI Testing Criteria

The generated response should be checked for:

- Relevance to the user's goal.
- Relationship to selected intensity.
- Day-wise organization.
- Workout details.
- Nutrition/recovery relevance.
- Response completeness.
- Successful display in the frontend.
- Successful storage when applicable.

Important Testing Consideration

Because the output is generated by an AI service, the exact wording of the response can vary between requests. Therefore, testing should focus on whether the response satisfies the required structure and purpose rather than requiring an identical text response every time.

20. Integration Testing

Integration testing checks whether individual components work correctly when connected together.

FitBuddy has four major layers:

FitBuddy- AI Fitness Plan Generator using Gemini Models

```

Frontend
↓
FastAPI Backend
↓
Gemini AI
↓
SQLite Database

```

Each connection should be tested.

Figure 6: FitBuddy Integration Testing

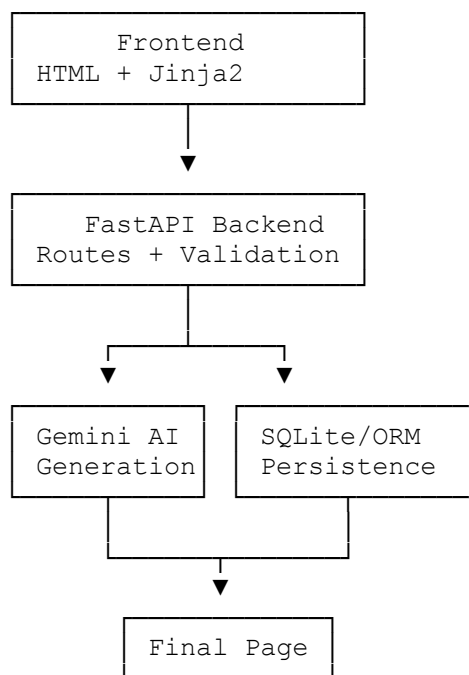


Figure 6: FitBuddy Integration Testing

21. End-to-End Testing

End-to-end testing verifies the complete application workflow from the user's first interaction until the final result.

Complete Test Scenario

1. Open FitBuddy.
2. Enter user name.
3. Enter user ID.
4. Enter age.
5. Enter weight.
6. Select fitness goal.
7. Select workout intensity.
8. Submit the form.
9. FastAPI receives the request.

10. AI generates the workout plan.
11. Nutrition/recovery advice is generated.
12. User and plan information is stored.
13. Result page displays the generated information.
14. User provides feedback.
15. Existing plan is retrieved.
16. AI revises the plan.
17. Updated plan is stored.
18. Updated result is displayed.
19. Admin view can display original and updated plans.

Figure 7: Complete End-to-End Test

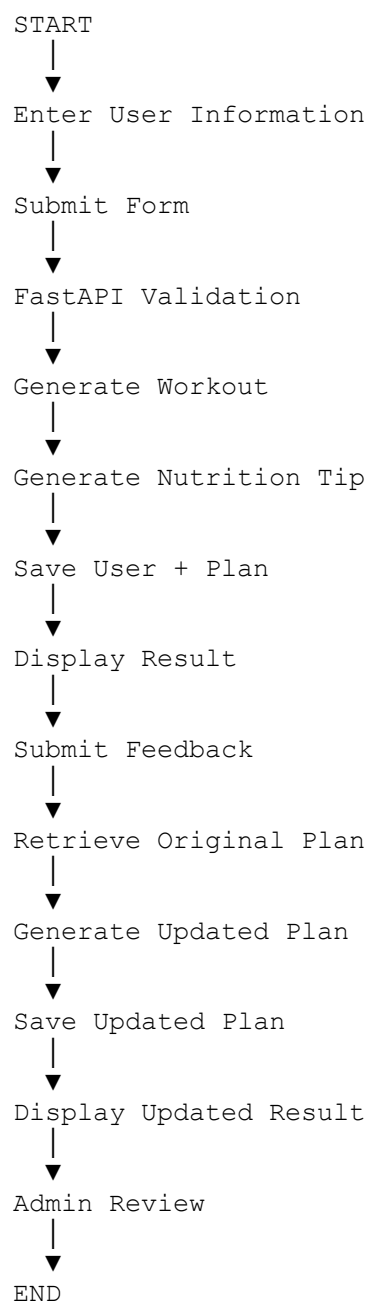


Figure 7: Complete FitBuddy End-to-End Testing Flow

22. Local Deployment Testing

The source project explicitly includes **Activity 5.2: Testing and Verifying Local Deployment**.

The application is intended to be started using Uvicorn and accessed through:

```
http://127.0.0.1:8000
```

The source also identifies:

```
http://127.0.0.1:8000/docs
```

as the FastAPI interactive documentation interface for exploring and testing available endpoints.

Deployment Test 01 – Server Startup

Expected Result:

The FastAPI application starts without a startup error.

Deployment Test 02 – Homepage

Expected Result:

The FitBuddy homepage loads in a browser.

Deployment Test 03 – API Documentation

Expected Result:

The `/docs` interface loads successfully.

Deployment Test 04 – Form Submission

Expected Result:

The homepage form can submit information to the backend.

Deployment Test 05 – Result Page

Expected Result:

The generated workout information is rendered through the result page.

23. Error and Exception Testing

Error testing is used to identify situations where the application may receive unexpected or incomplete information.

Potential conditions include:

- Empty user ID.
- Missing age.
- Missing weight.
- Missing fitness goal.
- Missing intensity.
- Invalid user ID during feedback.
- Feedback submitted without an existing plan.
- AI service unavailable.
- Invalid API key configuration.
- Database access problems.
- Template rendering problems.
- Server startup problems.

The purpose of these tests is to ensure that errors are handled in a controlled way rather than causing unexpected application behavior.

24. Test Case Summary

Test ID	Module	Test Description	Expected Result
UI-01	Input	Enter valid user details	Input accepted
UI-02	Input	Select fitness goal	Goal captured
UI-03	Input	Select intensity	Intensity captured
UI-04	Input	Submit incomplete form	Invalid/incomplete input handled
WG-01	Workout	Generate workout	Workout generated
WG-02	Workout	Check 7-day structure	Day-wise plan produced
WG-03	Workout	Goal-based generation	Goal considered
WG-04	Workout	Intensity-based generation	Intensity considered
NT-01	Nutrition	Generate tip	Tip generated
FB-01	Feedback	Submit feedback	Feedback accepted
FB-02	Feedback	Retrieve original plan	Original plan retrieved
FB-03	Feedback	Update plan	Revised plan generated
FB-04	Database	Save updated plan	Updated plan stored
DB-01	Database	Save user	User stored
DB-02	Database	Save plan	Plan stored
DB-03	Database	Retrieve plan	Plan retrieved
ADM-01	Admin	Display users	User records displayed
ADM-02	Admin	Display original plan	Original plan visible
ADM-03	Admin	Display updated plan	Updated plan visible

Test ID	Module	Test Description	Expected Result
RT-01	Routes	Open /	Homepage displayed
RT-02	Routes	Open /generate-workout workflow	Workout generated
RT-03	Routes	Submit feedback	Plan updated
RT-04	Routes	Open /view-all-users	Admin view displayed
DEP-01	Deployment	Start Uvicorn	Server starts
DEP-02	Deployment	Open localhost	Application loads
DEP-03	Deployment	Open /docs	API documentation loads

25. Requirement-to-Test Traceability

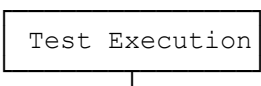
Testing should be connected to the requirements identified in the earlier phase.

Requirement	Related Test
User information collection	UI-01
Goal selection	UI-02
Intensity selection	UI-03
Personalized workout	WG-01
Seven-day plan	WG-02
Goal-based workout	WG-03
Nutrition guidance	NT-01
Feedback submission	FB-01
Plan modification	FB-03
Original plan preservation	FB-02 / FB-04
User storage	DB-01
Plan storage	DB-02
Plan retrieval	DB-03
Admin view	ADM-01
Route processing	RT-01 to RT-04
Local deployment	DEP-01 to DEP-03

26. Bug Identification and Resolution Process

If a problem is identified during testing, it should be recorded and tracked systematically.

Figure 8: Bug Lifecycle



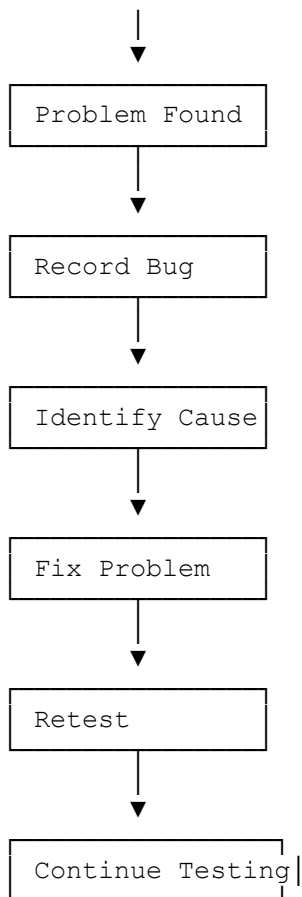


Figure 8: FitBuddy Bug Identification and Resolution Lifecycle

27. Testing the AI Feedback Loop

One of FitBuddy's important features is the feedback-based modification of an existing plan.

This feature should be tested as a complete loop.

Example

Original requirement:

User wants a workout plan.

Generated plan:

The system creates a seven-day plan.

User feedback:

"Add more cardio."

System processing:

The original plan and feedback are passed to the AI update function.

Expected output:

A revised plan reflecting the requested change.

FitBuddy- AI Fitness Plan Generator using Gemini Models

The source specifically documents this mechanism through `update_workout_plan()`, which receives the original plan and user feedback and returns a modified plan.

28. Testing the Database and AI Interaction

The application must also be checked for correct communication between the AI service and database.

Figure 9: AI and Database Testing

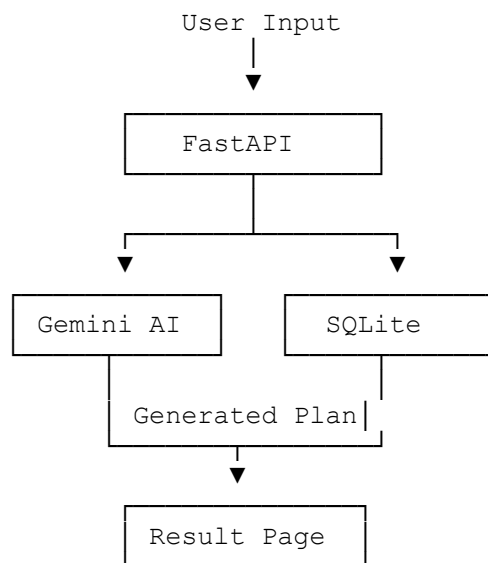


Figure 9: AI and Database Integration Testing

Testing should verify that the generated information is passed correctly from the AI layer to the backend and subsequently stored and displayed.

29. Testing Completion Criteria

The testing phase can be considered ready for completion when the following conditions have been checked:

- Main application starts successfully.
- Homepage is accessible.
- User input can be submitted.
- Workout generation workflow operates.
- Nutrition tip generation operates.
- Generated information reaches the result page.
- User information can be stored.
- Workout plans can be stored.
- Feedback can be submitted.
- Existing plans can be retrieved.
- Updated plans can be generated.

- Original and updated plans can be distinguished.
- Admin view can display stored information.
- FastAPI /docs is accessible.
- Frontend pages render correctly.
- Major integration paths have been checked.

30. Testing Limitations

The source FitBuddy document describes the local deployment verification process and the application's expected workflows, but it does **not provide a complete numerical test report containing actual pass/fail counts, defect counts, response-time measurements, or a formal test execution log.**

Therefore, those values should not be artificially added to the project documentation.

The test cases in this phase provide a structured testing framework based on the implemented FitBuddy functionality. Actual results can be entered after executing each test case on the student's local system.

31. Phase 6 Deliverables

The expected deliverables of the Project Testing Phase are:

1. Testing strategy.
2. Test environment description.
3. Functional test cases.
4. User-input test cases.
5. Workout-generation test cases.
6. Nutrition-tip test cases.
7. Feedback test cases.
8. Database test cases.
9. FastAPI route test cases.
10. Frontend/UI test cases.
11. AI response verification criteria.
12. Integration testing workflow.
13. End-to-end testing workflow.
14. Local deployment verification.
15. Bug identification process.
16. Requirement-to-test traceability.
17. Testing completion criteria.

32. Transition to Phase 7 – Project Documentation

After testing, the next phase is **Phase 7 – Project Documentation.**

The information collected during development and testing can be organized into a complete project document. This includes the project concept, requirements, architecture, development

activities, testing methodology, results, limitations, screenshots, diagrams, and final conclusions.

The documentation phase should present the project in a structured manner so that another person can understand how FitBuddy was designed, implemented, tested, and demonstrated.

33. Phase 6 Conclusion

The **Project Testing Phase** ensures that the FitBuddy – AI Fitness Plan Generator is systematically checked across its major components.

Testing covers the complete application flow from user input to AI-generated workout plans, nutrition or recovery guidance, database storage, feedback-based plan modification, frontend rendering, and local deployment.

FitBuddy's documented architecture connects the frontend, FastAPI backend, Gemini AI services, and SQLite database. Testing these components individually and together helps identify problems before the project is demonstrated or documented as complete.

The source project specifically includes local deployment verification through the Uvicorn server, the main application URL, and FastAPI's interactive `/docs` interface.